

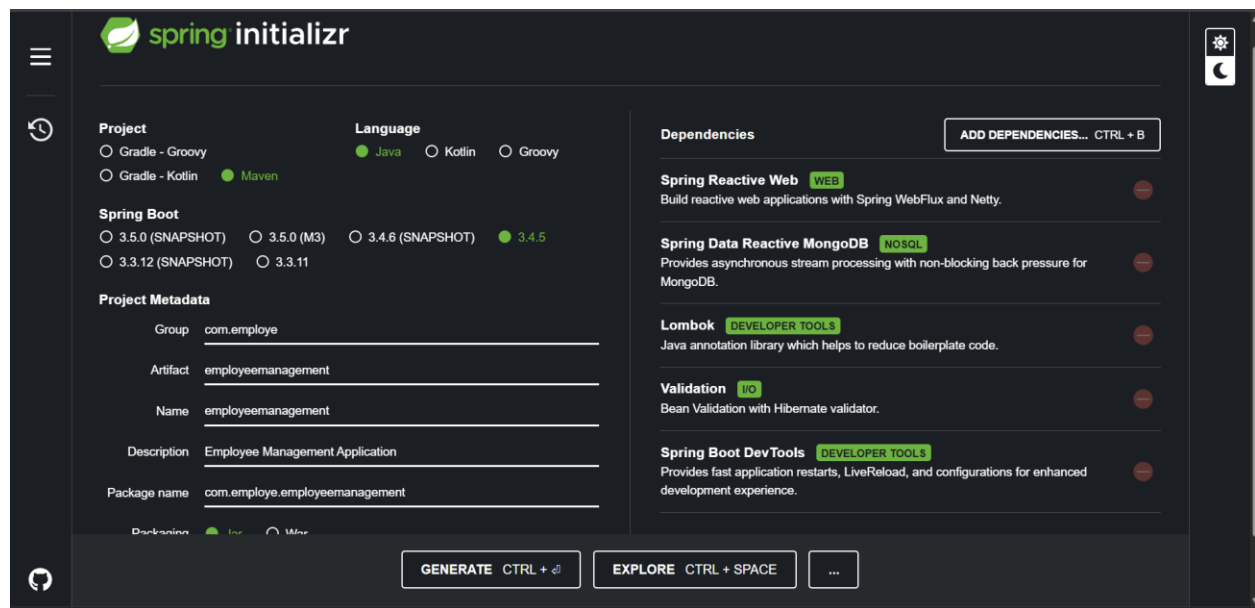
Employee Management System

Technologies & Tools

1	FRONTEND	HTML,CSS AND JS
2	DESIGN FRAMEWORK	BOOTSTRAP
3	BACKEND	SPRING BOOT (with spring webflux)
4	DATABASE	MONGODB
5	API TESTING	SWAGGER
6	TESTING	JUnit , Mockito
8	Documentation	PDF with ScreenShots

Features to Include (Minimum Requirements):

1. View All Employees
2. **+**Add New Employee
3. Update Employee Details
4. **X**Delete Employee
5. ☐ Search or Filter Employees
6. ☐ Validation (name, email, etc.)
7. Unit Testing
8. API Testing with Swagger
9. Final PDF Documentation with Screenshots



Employee Management System

Project Setup Plan (Employee Management System – using Fetch + MongoDB)

Step 1: Spring Boot Initializer Setup

1. Go to : <https://start.spring.io/>
2. **Project:** Maven
3. **Language:** Java
4. **Spring Boot Version:** Use latest stable
5. **Project Metadata:**
 1. Group: com.employee
 2. Artifact: employeemanagement
 3. Name: employeemanagement
 4. Package Name: com.employe.employeemanagement
6. **Dependencies:**
 1. Spring Reactive Web
 2. Spring Data Reactive MongoDB
 3. Lombok
 4. Validation
 5. Spring Boot DevTools
 6. Spring Security
 7. Spring Boot TestSpringdoc OpenAPI (Swagger UI)- `springdoc-openapi-starter-webflux-ui`
7. Then click **Generate**, unzip, and open in your IDE (IntelliJ, VS Code, etc.)

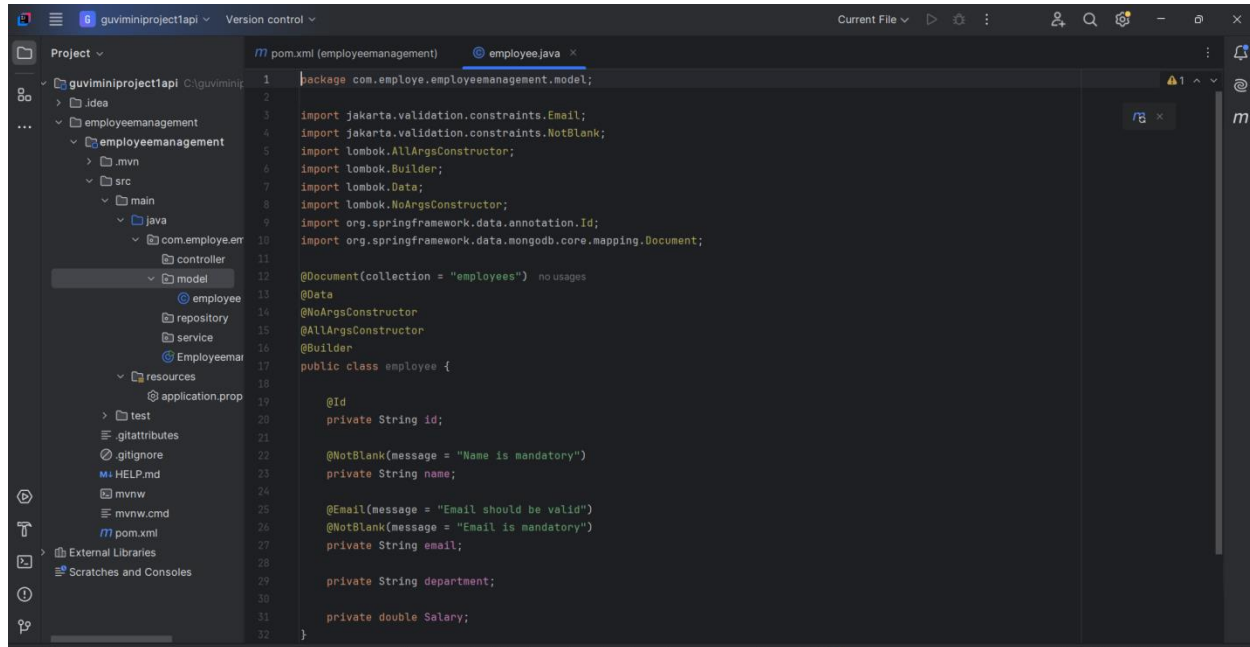
Step 2: Create packages

1. Model
2. Repository
3. Service
4. Controller
5. Config
6. Exception

Employee Management System

MODEL PACKAGE :

1. CLASS EMPLOYEE

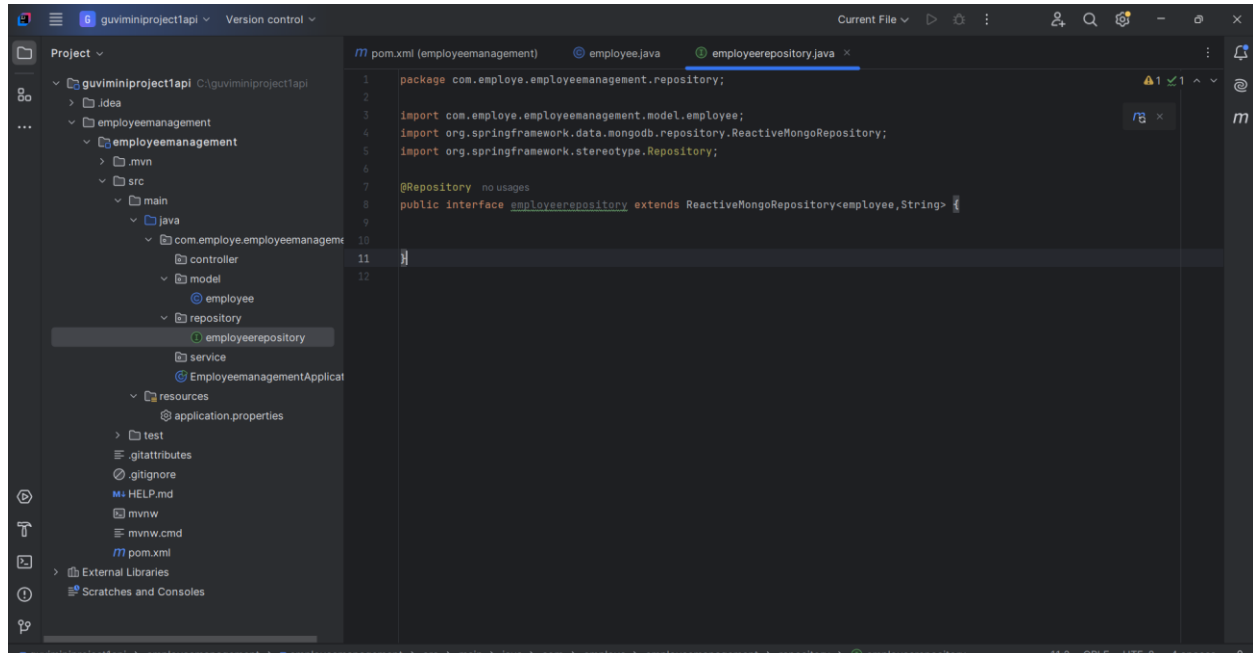


Annotation	Purpose
@Document(collection = "employees")	Tells Spring that this class maps to a MongoDB collection named employees.
@Data	Lombok annotation that creates all the boilerplate (getters, setters, toString(), equals(), etc.) automatically.
@NoArgsConstructor	Lombok creates a no-argument (default) constructor.
@AllArgsConstructor	Lombok creates a constructor with all fields as parameters.
@Builder	Helps you create objects in a clean, step-by-step way using the Builder pattern.
@NotBlank	Validates that the field is not null and not empty (it must contain at least one non-whitespace character). Mostly used for String fields.
@Email	Validates that the string provided is in a valid email format (like abc@example.com).

Employee Management System

REPOSITORY PACKAGE :

2. INTERFACE EMPLOYEE REPOSITORY



Explanation

- We extend **ReactiveMongoRepository**<employee, String>
 - employee → *the model NAME*
 - String → *the Id type*
- Spring Data automatically creates MongoDB queries.

SERVICE PACKAGE :

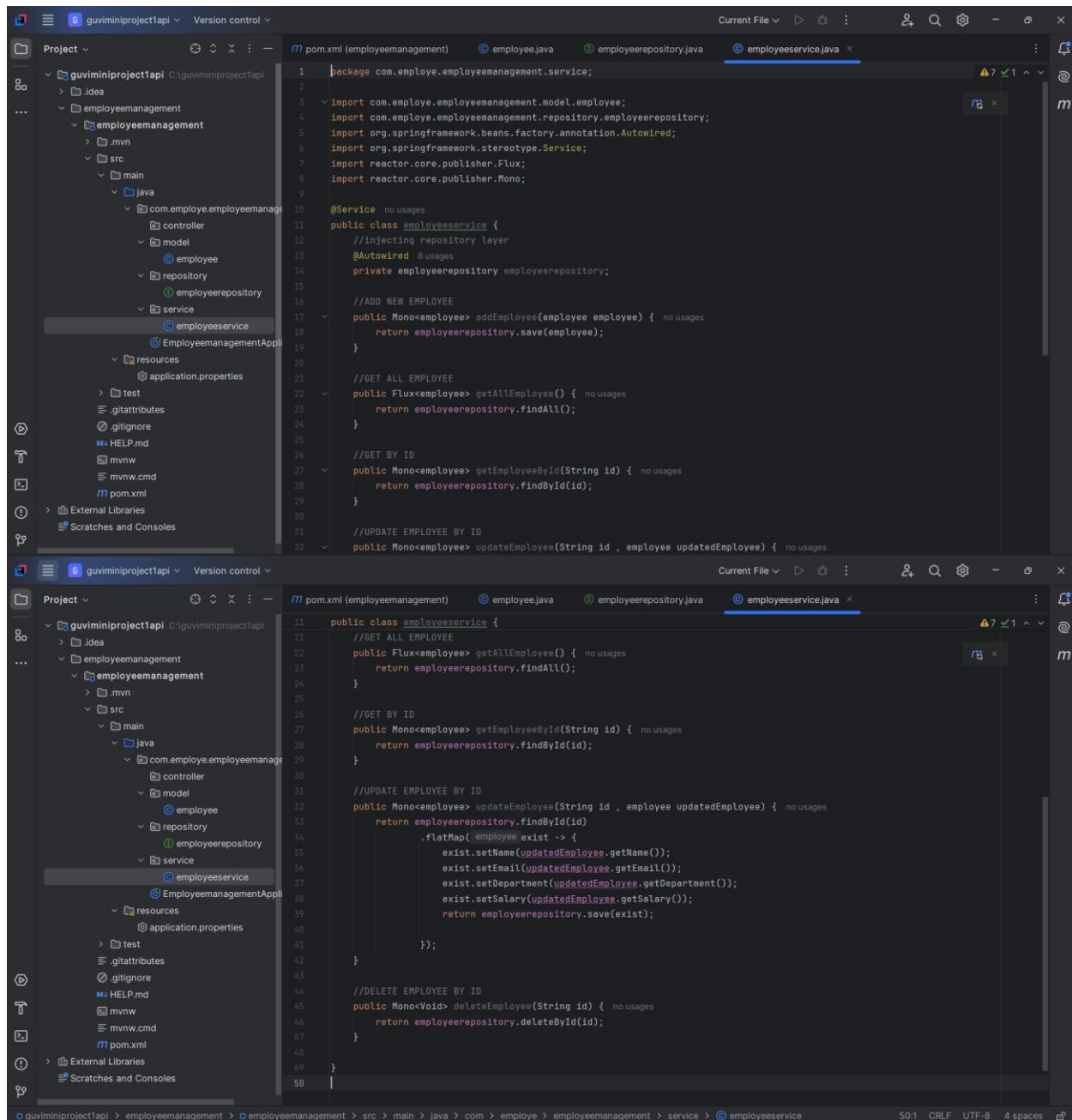
3. CLASS SERVICE

We'll use **Mono** and **Flux** from **Project Reactor**, since we're using **WebFlux**.

This layer contains the **business logic**:

- Add employee
- Get all employees
- Update employee
- Delete employee
- Get employee by ID

Employee Management System



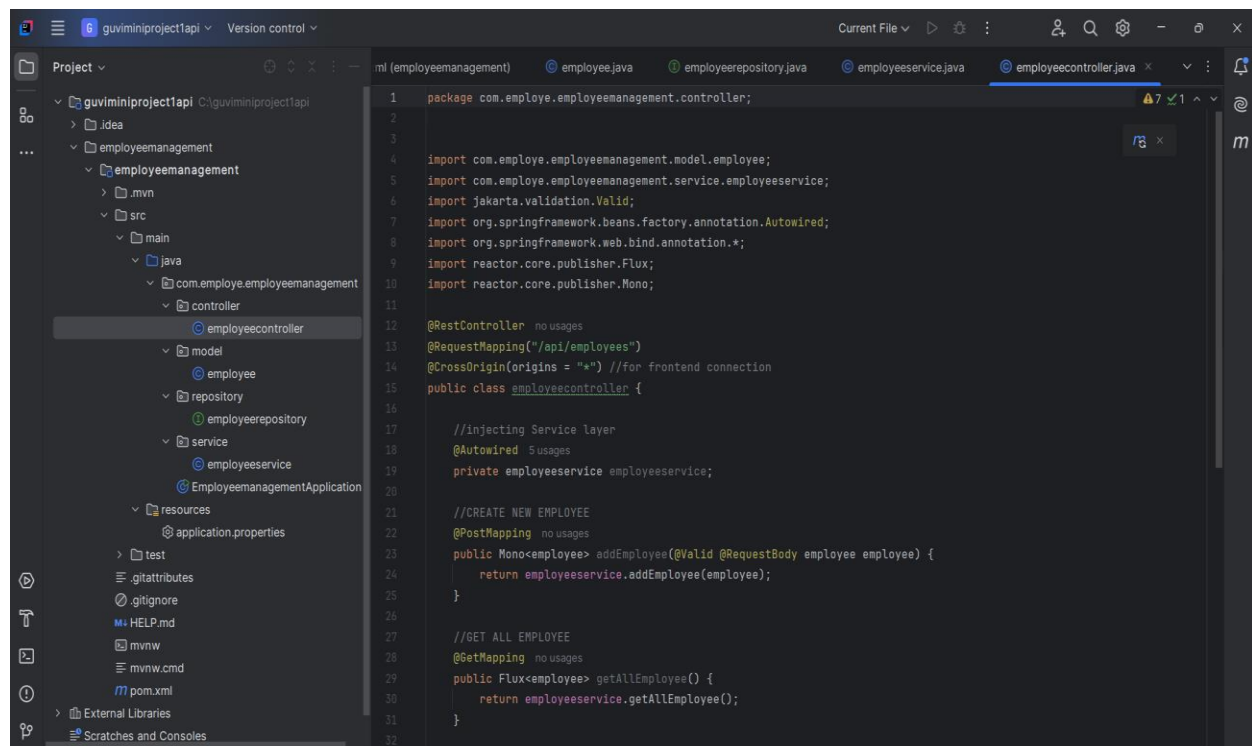
Employee Management System

Explanation

Method	Purpose
Mono<Employee>	Mono means single value (used for single employee)
Flux<Employee>	Flux means stream of multiple values (used for list of employees)
save()	Save to MongoDB
findById()	Find one By Id
findAll()	Get all
deleteById()	Delete by Id

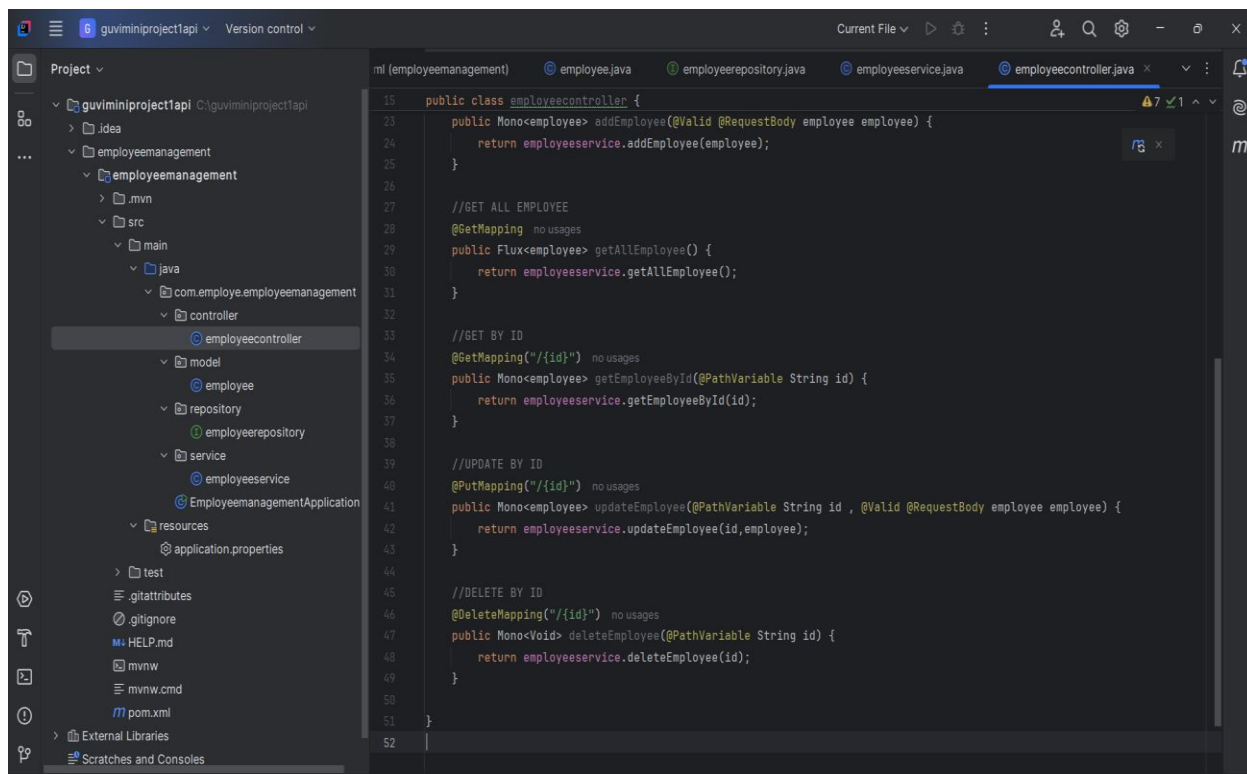
CONTROLLER PACKAGE :

4. CLASS EMPLOYEECONTROLLER



```
1 package com.employee.employeeemangement.controller;
2
3
4 import com.employee.employeeemangement.model.employee;
5 import com.employee.employeeemangement.service.employeeservice;
6 import jakarta.validation.Valid;
7 import org.springframework.beans.factory.annotation.Autowired;
8 import org.springframework.web.bind.annotation.*;
9 import reactor.core.publisher.Flux;
10 import reactor.core.publisher.Mono;
11
12 @RestController
13 @RequestMapping("/api/employees")
14 @CrossOrigin(origins = "*") //for frontend connection
15 public class employeecontroller {
16
17     //injecting Service layer
18     @Autowired
19     private employeeservice employeeservice;
20
21     //CREATE NEW EMPLOYEE
22     @PostMapping
23     public Mono<employee> addEmployee(@Valid @RequestBody employee employee) {
24         return employeeservice.addEmployee(employee);
25     }
26
27     //GET ALL EMPLOYEE
28     @GetMapping
29     public Flux<employee> getAllEmployee() {
30         return employeeservice.getAllEmployee();
31     }
32 }
```

Employee Management System



API ENDPOINTS SUMMARY

Operation	HTTP Method	API Path	What it Does
Add Employee	POST	/api/employees	Adds new employee
Get All	GET	/api/employees	Gets all employees
Get by ID	GET	/api/employees/{id}	Get one employee
Update	PUT	/api/employees/{id}	Update by ID
Delete	DELETE	/api/employees/{id}	Delete by ID

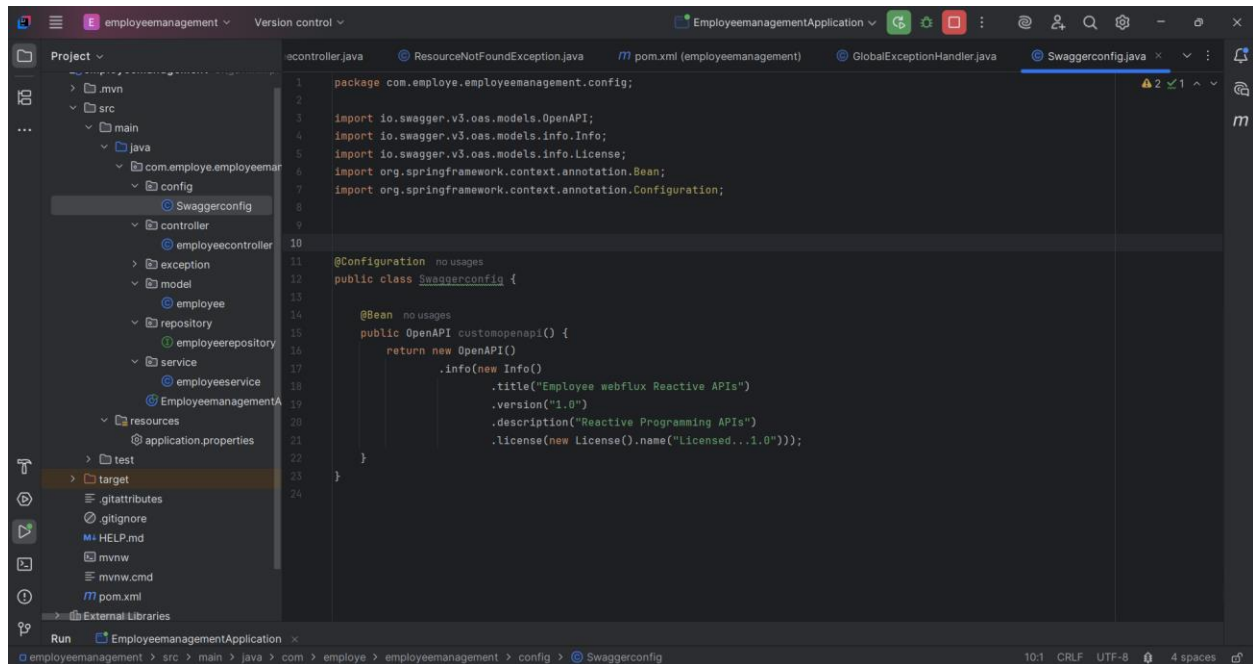
Annotations Used:

Annotation	Meaning (English)
@RestController	Marks this class as a REST API controller.
@RequestMapping("/api/employees")	All API URLs will start with /api/employees.
@CrossOrigin(origins = "*")	Allows requests from any frontend (CORS).
@Autowired	Spring will inject (auto-wire) the service here.
@PostMapping, @GetMapping, etc.	Define API methods for POST, GET, PUT, DELETE.

Employee Management System

CONFIG PACKAGE

SwaggerConfiguration class:



Purpose of this File

This file configures **Swagger/OpenAPI** for your Spring Boot application. Swagger helps to **automatically generate documentation** and a **UI page** to test your REST APIs easily.

@Configuration Annotation

1. @Configuration
 - This tells Spring Boot that this class provides **Spring configuration**.
 - Spring will scan and load this as part of the application context.

2.@Bean

```
public OpenAPI customopenapi() {
```

- This method returns a **custom OpenAPI configuration**
- @Bean means Spring will manage this object in its container.

Employee Management System

```
3. return new OpenAPI()

.info(new Info()

.title("Employee webflux Reactive APIs")

.version("1.0")

.description("Reactive Programming APIs")

.license(new License().name("Licensed...1.0")));
```

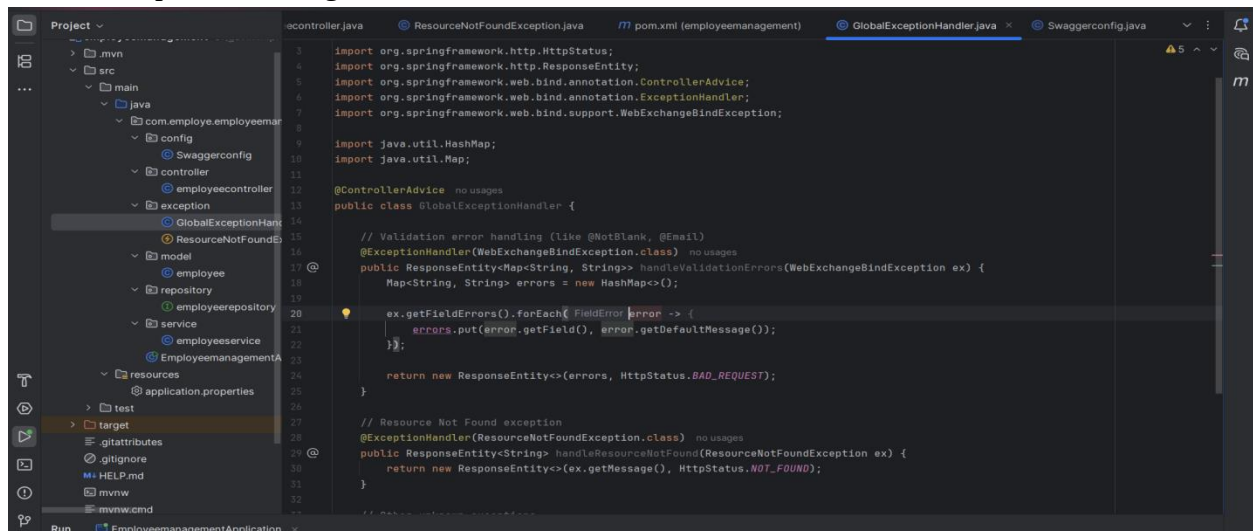
Method	Meaning (English)
title(...)	Title of the Swagger UI page
version(...)	Version number
description(...)	Small explanation of your API
license(...)	Any license info you want to show

So the output will be a **Swagger UI** page like:

Title: Employee webflux Reactive APIs
Version: 1.0
Description: Reactive Programming APIs
License: Licensed...1.0

EXCEPTION PACKAGE

GlobalExceptionHandler class



Employee Management System

This code handles all types of errors (validation, not found, unknown errors) in one central place using `@ControllerAdvice`.

`@ControllerAdvice`

```
public class GlobalExceptionHandler {
```

`@ControllerAdvice`: Special Spring annotation to handle exceptions globally across the whole application.

1. Validation Error Handling

```
@ExceptionHandler(WebExchangeBindException.class)
public ResponseEntity<Map<String, String>>
handleValidationErrors(WebExchangeBindException ex) {
    Map<String, String> errors = new HashMap<>();

    ex.getFieldErrors().forEach(error -> {
        errors.put(error.getField(), error.getDefaultMessage());
    });

    return new ResponseEntity<>(errors, HttpStatus.BAD_REQUEST);
}
```

- This method handles **backend validation errors** (like `@NotBlank`, `@Email`, `@Min`).
- Catches the validation errors from WebFlux (that's why it uses `WebExchangeBindException`).
- For each invalid field, it creates a map:

Return:

- HTTP Status: 400 BAD REQUEST
- Body: Field name → Error message

2.Resource Not Found Handling

```
@ExceptionHandler(ResourceNotFoundException.class)
public ResponseEntity<String> handleResourceNotFound(ResourceNotFoundException
ex) {
    return new ResponseEntity<>(ex.getMessage(), HttpStatus.NOT_FOUND);
}
```

- If employee ID not found in DB, we throw `ResourceNotFoundException`.
- This method catches it and returns a 404 error with a message

Employee Management System

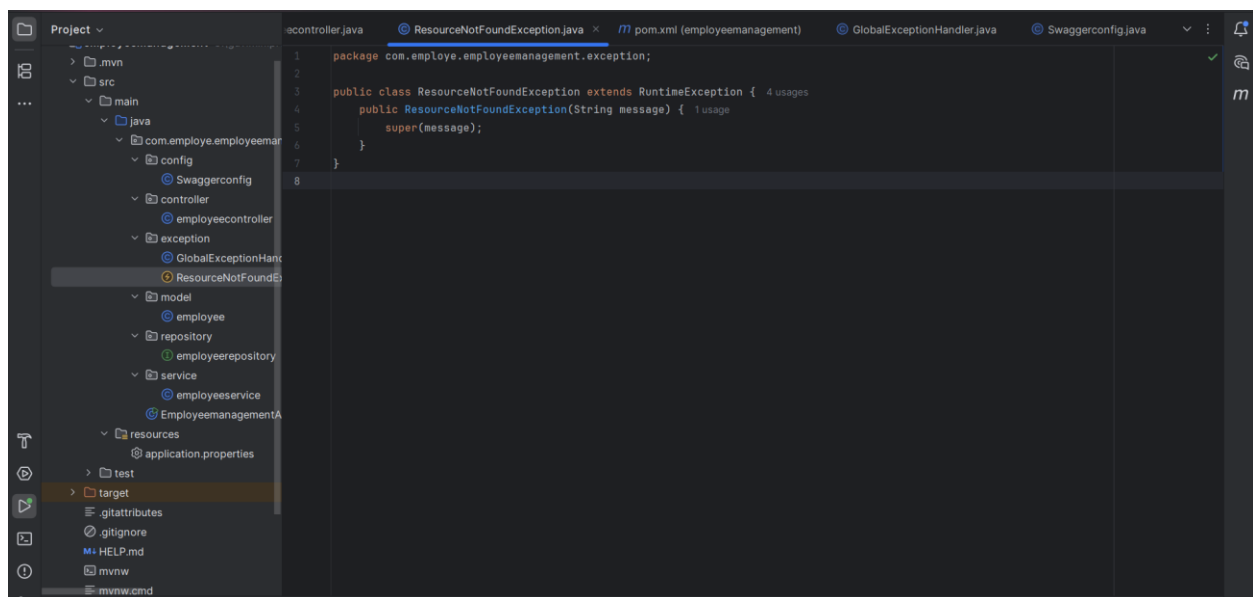
3.General Exception (Fallback for all other errors)

@ExceptionHandler(Exception.class)

```
public ResponseEntity<String> handleGeneral(Exception ex) {  
  
    return new ResponseEntity<>("Internal Server Error: " + ex.getMessage(),  
HttpStatus.INTERNAL_SERVER_ERROR);  
  
}
```

- This is the fallback for **any unhandled exceptions**.
- Useful for debugging and preventing server crashes.
- Returns a 500 Internal Server Error.

FILE 2 :ResourceNotFoundException.java



```
public class ResourceNotFoundException extends RuntimeException {  
    public ResourceNotFoundException(String message) {  
        super(message);  
    }  
}
```

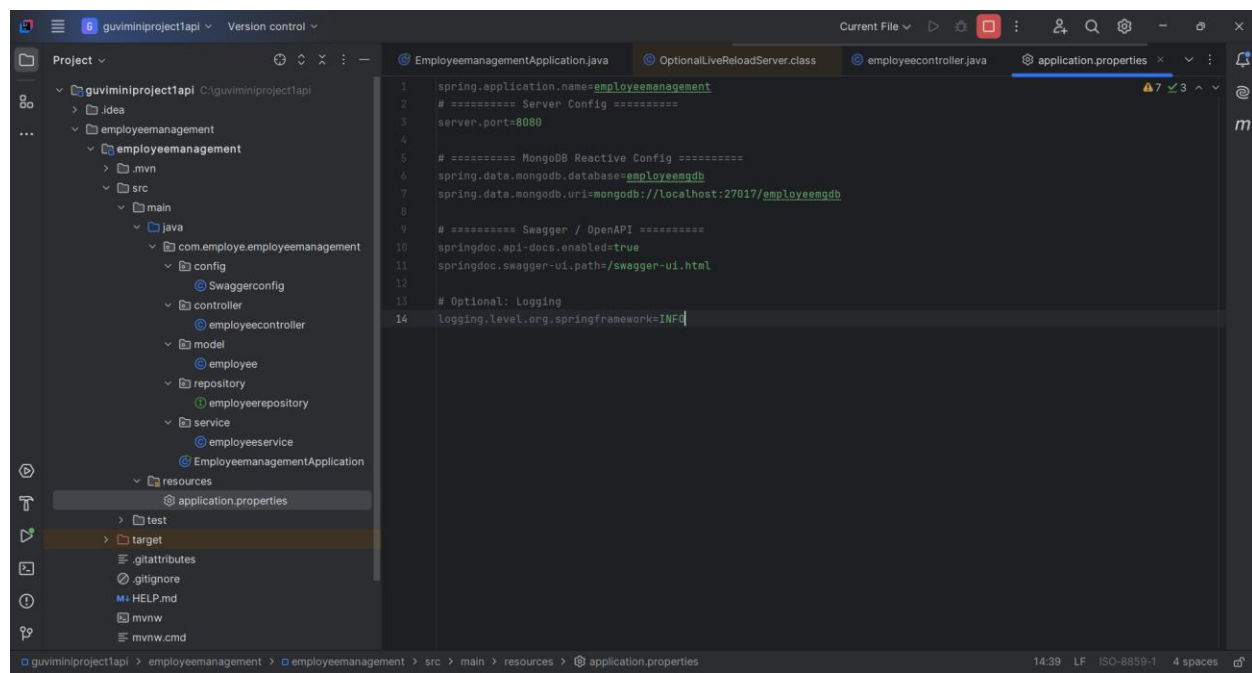
- This is a **custom exception**.
- We use it to throw errors when a specific employee is not found.

Employee Management System

Exception Type	Caught By	HTTP Status	Description
Validation errors	WebExchangeBindException	400 (Bad Request)	Form validation miss
Resource not found	ResourceNotFoundException	404 (Not Found)	IF EMPLOYEE ID NOT PRESENT
General/Other exceptions	Exception	500 (Internal Server Error)	Unknown issue

APPLICATION PROPERTIES :

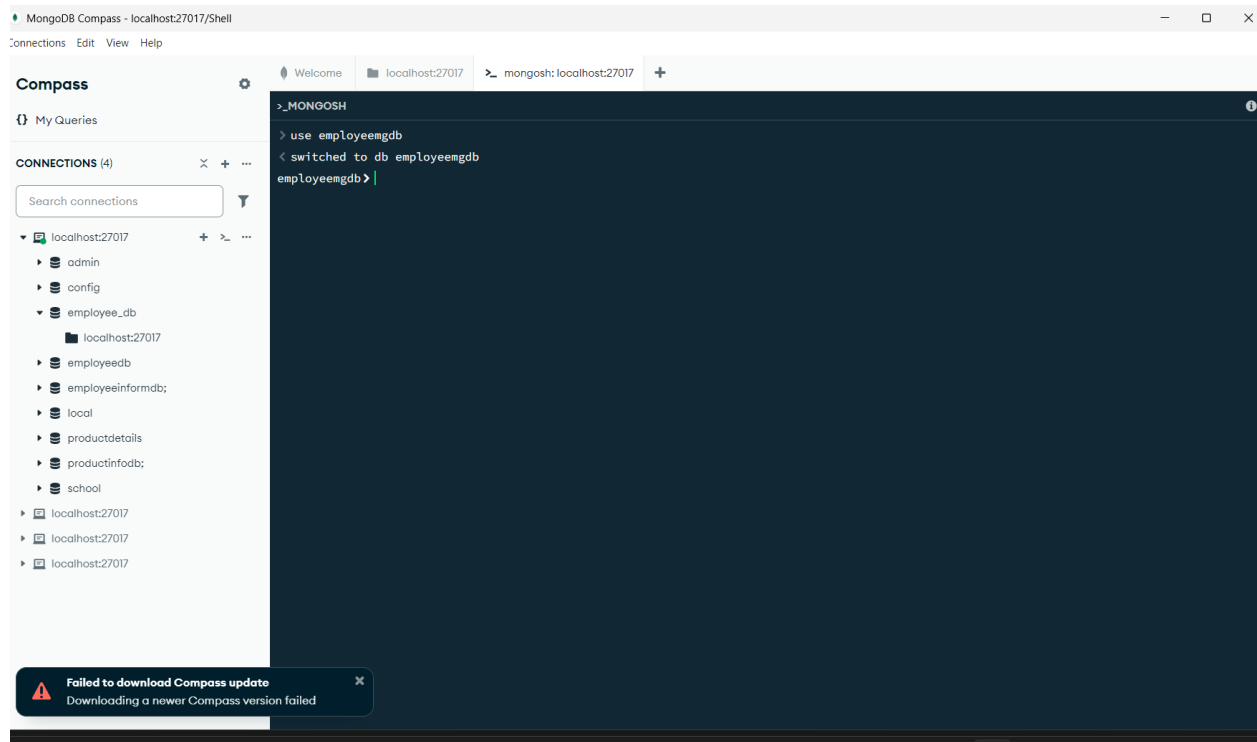
Property	Meaning
server.port=8080	Starts your backend server on port 8080
spring.data.mongodb.uri	Tells Spring where your MongoDB is running
springdoc.swagger-ui.path	Enables Swagger UI at http://localhost:8080/swagger-ui.html



```
1 spring.application.name=employeemanagement
2 # ===== Server Config =====
3 server.port=8080
4
5 # ===== MongoDB Reactive Config =====
6 spring.data.mongodb.database=employeedb
7 spring.data.mongodb.uri=mongodb://localhost:27017/employeedb
8
9 # ===== Swagger / OpenAPI =====
10 springdoc.api-docs.enabled=true
11 springdoc.swagger-ui.path=/swagger-ui.html
12
13 # Optional: Logging
14 logging.level.org.springframework=INFO
```

Employee Management System

Now start MongoDB



INSERT DATABASE “employeeemgdb”

Then run your Spring Boot app and go to:

<http://localhost:8080/swagger-ui.html>

You'll see all your APIs displayed neatly.

API Testing using Swagger (OpenAPI)

If you followed Step 5, Swagger should already be enabled.

Test URL:

After running the backend, open your browser and visit: <http://localhost:8080/swagger-ui.html>

You will see:

- POST /api/employees → Add Employee
- GET /api/employees → Get All
- PUT /api/employees/{id} → Update
- DELETE /api/employees/{id} → Delete
- GET /api/employees/{id} → Get One

Employee Management System

You can click “**Try it out**”, enter request data, and see live responses ✓

Employee webflux Reactive APIs 1.0 OAS 3.0

[v3/api-docs](#)

Reactive Programming APIs

Licensed...1.0

Servers

http://localhost:8080 - Generated server url

employeecontroller

GET

/api/employees/{id}

PUT

/api/employees/{id}

DELETE

/api/employees/{id}

GET

/api/employees

POST

/api/employees

POST

POST /api/employees

Parameters

No parameters

Request body required

application/json

```
{
  "name": "BANUPRIYA",
  "email": "banu@gmail.com",
  "department": "DEVELOPER",
  "salary": 50000
}
```

Execute

Clear

Responses

Curl

```
curl -X POST \
```

Employee Management System

RESPONSE

The screenshot shows a REST client interface with the following details:

- Method:** POST
- Request URL:** http://localhost:8080/api/employees
- Request Headers:**

```
curl -X 'POST' \
  'http://localhost:8080/api/employees' \
  -H 'accept: */*' \
  -H 'Content-Type: application/json' \
  -d '{
    "name": "BANUPRIYA",
    "email": "banu@gmail.com",
    "department": "DEVELOPER",
    "salary": 50000
  }'
```
- Response Code:** 200
- Response Body:**

```
{
  "id": "688be90ff063245f940545f4",
  "name": "BANUPRIYA",
  "email": "banu@gmail.com",
  "department": "DEVELOPER",
  "salary": 50000
}
```
- Response Headers:**

```
content-length: 119
content-type: application/json
vary: Origin,Access-Control-Request-Method,Access-Control-Request-Headers
```

DATABASE CONNECTION

The screenshot shows the MongoDB Compass interface with the following details:

- Database:** employeeemgdb
- Collection:** employees
- Documents:** 1 document is displayed.
- Document Data:**

```
{
  "_id": ObjectId('688be90ff063245f940545f4'),
  "name": "BANUPRIYA",
  "email": "banu@gmail.com",
  "department": "DEVELOPER",
  "Salary": 50000,
  "_class": "com.employe.employeeemangement.model.employee"
}
```

Employee Management System

GET

The screenshot shows a REST client interface with the following details:

- Method:** GET
- URL:** /api/employees
- Parameters:** No parameters
- Execute:** Button to execute the request
- Clear:** Button to clear the request
- Responses:** Section showing the response details
- Curl:**

```
curl -X 'GET' \
  'http://localhost:8080/api/employees' \
  -H 'accept: */*'
```
- Request URL:** http://localhost:8080/api/employees
- Server response:** 200
- Code:** 200
- Details:** Response body
- Response body:**

```
{
  "id": "688be90ff063245f940545f4",
  "name": "BANUPRIYA",
  "email": "banu@gmail.com",
  "department": "DEVELOPER",
  "salary": 50000
},
{
  "id": "688bea72f063245f940545f5",
  "name": "ROSEKUNAR",
  "email": "rose@gmail.com",
  "department": "TESTING",
  "salary": 60000
},
{
  "id": "688bea9df063245f940545f6",
  "name": "SHASHINI",
  "email": "shashini@gmail.com",
  "department": "PRODUCT OWNER",
  "salary": 100000
},
{
  "id": "688beacaf063245f940545f7",
  "name": "YASHKUNAR",
  "email": "yash@gmail.com",
  "department": "SCRUM MASTER",
  "salary": 80000
}
```

RESPONSE

The screenshot shows the response details in a REST client interface:

- Response body:**

```
{
  "id": "688be90ff063245f940545f4",
  "name": "BANUPRIYA",
  "email": "banu@gmail.com",
  "department": "DEVELOPER",
  "salary": 50000
},
{
  "id": "688bea72f063245f940545f5",
  "name": "ROSEKUNAR",
  "email": "rose@gmail.com",
  "department": "TESTING",
  "salary": 60000
},
{
  "id": "688bea9df063245f940545f6",
  "name": "SHASHINI",
  "email": "shashini@gmail.com",
  "department": "PRODUCT OWNER",
  "salary": 100000
},
{
  "id": "688beacaf063245f940545f7",
  "name": "YASHKUNAR",
  "email": "yash@gmail.com",
  "department": "SCRUM MASTER",
  "salary": 80000
}
```
- Response headers:**

```
content-type: application/json
transfer-encoding: chunked
vary: Origin,Access-Control-Request-Method,Access-Control-Request-Headers
```
- Responses:** Table with 3 columns: Code, Description, Links
- Code:** 200
- Description:** OK
- Links:** No links
- Media type:** */*
- Controls:** Accept header
- Example Value:** Schema

Employee Management System

GET BY ID

GET

/api/employees/{id}

Cancel

Parameters

Name	Description
id required	
string	680bea9df063245f940545f6
(path)	

Execute

Clear

Responses

Curl

curl -X 'GET' \n'http://localhost:8080/api/employees/680bea9df063245f940545f6' \n-H 'accept: */*'

Request URL

http://localhost:8080/api/employees/680bea9df063245f940545f6

Server response

Code	Details
200	Response body {\n "id": "680bea9df063245f940545f6",\n "name": "SHASHINI",\n}

TOTAL EMPLOYEE LIST

localhost:8080/api/employees

Pretty-print

```
[
  {
    "id": "680be90ff063245f940545f4",
    "name": "BANUPRIYA",
    "email": "banu@gmail.com",
    "department": "DEVELOPER",
    "salary": 50000
  },
  {
    "id": "680bea72f063245f940545f5",
    "name": "ROSEKUMAR",
    "email": "rose@gmail.com",
    "department": "TESTING",
    "salary": 60000
  },
  {
    "id": "680bea9df063245f940545f6",
    "name": "SHASHINI",
    "email": "shashini@gmail.com",
    "department": "PRODUCT OWNER",
    "salary": 100000
  },
  {
    "id": "680beacaf063245f940545f7",
    "name": "YASHKUMAR",
    "email": "yash@gmail.com",
    "department": "SCRUM MASTER",
    "salary": 80000
  },
  {
    "id": "680beb05f063245f940545f8",
    "name": "KAMALI",
    "email": "kamali@gmail.com",
    "department": "MERNSTACK DEVELOPER",
    "salary": 50000
  },
  {
    "id": "680beb2cf063245f940545f9",
    "name": "CHELLAM",
    "email": "chellam@gmail.com",
    "department": "TESTING",
    "salary": 60000
  },
  {
    "id": "680beb5df063245f940545fa",
    "name": "ANANTHAN"
  }
]
```

Employee Management System

DELETE

DELETE /api/employees/{id}

Parameters

Name	Description
id * required	
string (path)	680beb05f063245f940545f8

Execute Clear

Responses

Curl

```
curl -X 'DELETE' \
  'http://localhost:8080/api/employees/680beb05f063245f940545f8' \
  -H 'accept: */*'
```

Request URL

```
http://localhost:8080/api/employees/680beb05f063245f940545f8
```

Server response

Code	Details
200	Response headers content-length: 0 vary: Origin,Access-Control-Request-Method,Access-Control-Request-Headers

First — Overall Structure of Unit Test:

In any Unit Test, we always follow 4 steps:

Step	What We Do	Why?
1. Mock	Fake/mock the backend/database behavior	So test will not touch real database
2. Action	Call the service method you want to test	Main method we are checking
3. Verify Data	Check the result using StepVerifier	Confirm if output is correct
4. Verify Behavior	Check if repository method was called properly	Confirm method flow

Test 1: Get All Employees — testGetAllEmployees()

```
29 //GET ALL EMPLOYEE
30 @Test
31 void testGetAllEmployees() {
32     employee emp1 = new employee(id: "1", name: "Priya", email: "priya@gmail.com", department: "HR", salary: 50000);
33     employee emp2 = new employee(id: "2", name: "Kumar", email: "kumar@gmail.com", department: "IT", salary: 60000);
34
35     when(employeeRepository.findAll()).thenReturn(Flux.just(emp1, emp2));
36
37     Flux<employee> result = employeeservice.getAllEmployee();
38
39     StepVerifier.create(result)
40         .expectNext(emp1)
41         .expectNext(emp2)
42         .verifyComplete();
43
44     verify(employeeRepository, times(wantedNumberOfInvocations: 1)).findAll();
45 }
```

Employee Management System

EXPLANATION :TEST 1

Step	Code	Meaning
1. Mock	<code>when(employeeRepository.findAll()).thenReturn(Flux.just(emp1, emp2));</code>	If someone calls <code>findAll</code> , return two employees (Priya and Rose)
2. Action	<code>Flux<employee> result = employeeservice.getAllEmployees();</code>	Call the service method to get all employees
3. Verify Data	<code>StepVerifier.create(result).expectNext(emp1).expectNext(emp2).verifyComplete();</code>	Check that first Priya, then Rose are returned
4. Verify Behavior	<code>verify(employeeRepository, times(1)).findAll();</code>	Confirm that <code>findAll</code> method was called exactly once

Test 2: CREATE NEW EMPLOYEE — `testCreateEmployee()`;

```
46 // POST METHOD
47 @Test
48 void testCreateEmployee() {
49     employee employee = new employee( id: "1", name: "Priya", email: "priya@gmail.com", department: "HR", Salary: 50000);
50
51     when(employeeRepository.save(employee)).thenReturn(Mono.just(employee));
52
53     Mono<employee> result = employeeservice.createEmployee(employee);
54
55     StepVerifier.create(result) FirstStep<employee>
56         .expectNext(employee) Step<employee>
57         .verifyComplete();
58
59     verify(employeeRepository, times( wantedNumberOfInvocations: 1)).save(employee);
60 }
```

Employee Management System

Step	Code	Meaning
1. Mock	<code>when(employeeRepository.save(employee)).thenReturn(Mono.just(employee));</code>	If someone saves an employee, return the same employee (fake)
2. Action	<code>Mono<employee> result = employeeservice.createEmployee(employee);</code>	Call service method to save an employee
3. Verify Data	<code>StepVerifier.create(result).expectNext(employee).verifyComplete();</code>	Check if the result gives the same employee correctly
4. Verify Behavior	<code>verify(employeeRepository, times(1)).save(employee);</code>	Confirm that the save method was called exactly once

Test 3: GET THE ELEMENT BY ID — `testGetEmployeeById()`

```

61 //get by {id}
62 @Test
63 void testGetEmployeeById() {
64     String employeeId = "1";
65     employee employee = new employee(employeeId, name: "Priya", email: "priya@gmail.com", department: "HR", Salary: 50000);
66
67     when(employeeRepository.findById(employeeId)).thenReturn(Mono.just(employee));
68
69     Mono<employee> result = employeeservice.getEmployeeById(employeeId);
70
71     StepVerifier.create(result)
72         .expectNext(employee)
73         .verifyComplete();
74
75     verify(employeeRepository, times(wantedNumberOfInvocations: 1)).findById(employeeId);
76 }

```

Step	Code	Meaning
1. Mock	<code>when(employeeRepository.findById(employeeId)).thenReturn(Mono.just(employee));</code>	If someone searches for ID 1 give employee priya
2. Action	<code>Mono<employee> result = employeeservice.getEmployeeById(employeeId);</code>	Call service method to get employee by ID

Employee Management System

3. Verify Data	StepVerifier.create(result).expectNext(employee).verifyComplete();	Check if the correct employee is returned
4. Verify Behavior	verify(employeeRepository, times(1)).findById(employeeId);	Confirm that the findById method was called exactly once

Test 3: DELETE EMPLOYEE BY ID — testDeleteEmployee()

```
76 //delete by id
77 @Test
78 void testDeleteEmployee() {
79     String employeeId = "1";
80
81     when(employeeRepository.deleteById(employeeId)).thenReturn(Mono.empty());
82
83     Mono<Void> result = employeeservice.deleteEmployee(employeeId);
84
85     StepVerifier.create(result)
86         .verifyComplete();
87
88     verify(employeeRepository, times(wantedNumberOfInvocations: 1)).deleteById(employeeId);
89 }
90
91
92
93
```

Step	Code	Meaning
1. Mock	when(employeeRepository.deleteById(employeeId)).thenReturn(Mono.just(employee));	If someone delete for ID 1 ,just return empty(nothing)
2. Action	Mono<void> result = employeeservice.deleteEmployee(employeeId);	Call service method to delete employee by ID
3. Verify Data	StepVerifier.create(result).verifyComplete();	No data should return –just complete.

Employee Management System

4. Verify Behavior

```
verify(employeeRepository,  
times(1)).deleteById(employeeId);
```

Confirm that the deleteById method was called exactly once

Important Concepts

Concept	Simple Meaning
Mono	Single value async (like promise)
Flux	Multiple values async (like list of promises)
Mockito when()	Create fake behaviors
Mockito verify()	Confirm correct methods were called
StepVerifier	Testing reactive flows

TEST RESULTS

```
import com.employee.employeeManagement.service.employeeService;  
import org.junit.jupiter.api.Test;  
import org.mockito.InjectMocks;  
import org.mockito.Mock;  
import org.springframework.boot.test.context.SpringBootTest;  
import reactor.core.publisher.Flux;  
import reactor.core.publisher.Mono;  
import reactor.test.StepVerifier;  
  
import static org.mockito.Mockito.when;  
  
@SpringBootTest  
class EmployeeManagementApplicationTests {  
  
    @Mock  
    private employeeRepository employeeRepository;  
  
    @InjectMocks  
    private employeeService employeeService;  
  
    @Test  
    void testDeleteEmployee() {  
        // ...  
    }  
  
    @Test  
    void testGetEmployeeById() {  
        // ...  
    }  
  
    @Test  
    void testCreateEmployee() {  
        // ...  
    }  
  
    @Test  
    void testGetAllEmployees() {  
        // ...  
    }  
}
```

Run EmployeeManagementApplicationTests

Tests passed: 4 of 4 tests - 155 ms

19:10:56.363 [main] INFO org.springframework.test.context.support.AnnotationConfigContextLoaderUtils -- Could not detect default configuration classes for class [EmployeeManagementApplicationTests] in package [com.employee.employeeManagementApplicationTests]: EmployeeManagementApplicationTests does not define any static final fields for annotation [org.springframework.test.context.annotations.ConfigurationClass].

19:10:56.571 [main] INFO org.springframework.boot.test.context.SpringBootTest -- Found @SpringBootTestConfiguration com.employee.employeeManagementApplicationTests

Mockito is currently self-attaching to enable the inline mock-maker. This will no longer work in future releases of the JDK. Please add Mockito to the classpath of your application.

WARNING: A Java agent has been loaded dynamically (C:\Users\banup\.m2\repository\net\bytebuddy\byte-buddy-agent\1.15.11\byte-buddy-agent-1.15.11.jar). If a serviceability tool is in use, please run with -XX:-EnableDynamicAgentLoading to hide this warning.

WARNING: If a serviceability tool is not in use, please run with -Djdk.instrument.traceUsage for more information.

All the test cases are passed

Employee Management System

FRONTEND DEVELOPMENT

HTML,CSS,JAVASCRIPT ALSO USING BOOTSTRAP

In it we using fetch method to fetch the data from the database

HTML FILE

```
File Edit Selection View Go Run Terminal Help
index.html X JS script.js
index.html > html > body > div.container.my-5 > form#employeeForm > div.mb-3
2 <html lang="en">
34 </body>
36 <!-- Navbar -->
37 <nav class="navbar navbar-expand-lg bg-green">
38 <div class="container">
39 <a class="navbar-brand text-white fw-bold" href="#">Employee Management</a>
40 </div>
41 </nav>
42
43 <!-- Add/Edit Form -->
44 <div class="container my-5">
45 <h3 id="formTitle">Add Employee</h3>
46 <form id="employeeForm">
47 <div class="mb-3">
48 <label for="name" class="form-label">Name</label>
49 <input type="text" id="name" class="form-control" placeholder="Enter Name" required>
50 <div class="text-danger" id="nameError"></div>
51 </div>
52 <div class="mb-3">
53 <label for="email" class="form-label">Email</label>
54 <input type="email" id="email" class="form-control" placeholder="Enter Email" required>
55 <div class="text-danger" id="emailError"></div>
56 </div>
57 <div class="mb-3">
58 <label for="department" class="form-label">Department</label>
59 <input type="text" id="department" class="form-control" placeholder="Enter Department" required>
60 <div class="text-danger" id="departmentError"></div>
61 </div>
62 <div class="mb-3">
63 <label for="salary" class="form-label">Salary</label>
64 <input type="number" id="salary" class="form-control" placeholder="Enter Salary" required>
65 <div class="text-danger" id="salaryError"></div>
66 </div>
67 <button type="submit" class="btn btn-green" id="submitBtn">Add Employee</button>
68 </form>
69 </div>
70
```

IN LINE STYLE WITH BOOTSTRAP

```
File Edit Selection View Go Run Terminal Help
index.html X JS script.js
index.html >
1 <!DOCTYPE html>
2 <html lang="en">
3 <head>
4 <meta charset="UTF-8">
5 <meta name="viewport" content="width=device-width, initial-scale=1">
6 <title>Employee Management</title>
7 <link href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.3/dist/css/bootstrap.min.css" rel="stylesheet">
8 <style>
9 body {
10 background-color: #e8f5e9;
11 font-family: 'Segoe UI', sans-serif;
12 }
13 .bg-green {
14 background-color: #43a047 !important;
15 }
16 .btn-green {
17 background-color: #2e7d32;
18 color: white;
19 }
20 .btn-green:hover {
21 background-color: #1b5e20;
22 }
23 .form-control:invalid {
24 border-color: #dc3545;
25 }
26 .text-danger {
27 font-size: 0.875em;
28 }
29 .container h3 {
30 color: #2e7d32;
31 }
32 </style>
33 </head>
34 <body>
35
36 <!-- Navbar -->
37 <nav class="navbar navbar-expand-lg bg-green">
```

Employee Management System

JAVASCRIPT FILE

1.Global Declaration

```
JS script.js > ...
1  const apiUrl = 'http://localhost:8080/api/employees';
2  
3  let editingId = null;
4
```

- apiUrl: The base URL to call your Spring Boot backend API.
- editingId: Tracks the current employee being edited. It's null if you're adding a new employee.

2.DOMContentLoaded Event

```
5  document.addEventListener('DOMContentLoaded', () => {
6    fetchEmployees();
7    document.getElementById('employeeForm').addEventListener('submit', handleFormSubmit);
8  });
9
```

- This runs after the HTML page is fully loaded.
- Calls fetchEmployees() to load employee data from the backend.
- Adds a submit event listener to the form to handle add/update.

3.Loader Toggle

```
10 // Show loader
11 function toggleLoader(show) {
12   document.getElementById('loader').style.display = show ? 'block' : 'none';
13 }
14
```

- Shows or hides a loading spinner based on the show flag (true or false).

4.Fetch All Employees

- Fetches all employees from the backend API.
- Builds and displays the employee table dynamically.
- Shows a loader during fetch and handles errors gracefully.

Employee Management System

```
14
15 // Fetch the employee data
16 function fetchEmployees() {
17   toggleLoader(true);
18   fetch(apiUrl)
19     .then(res => res.json())
20     .then(data => {
21       const tbody = document.querySelector('#employeeTable tbody');
22       tbody.innerHTML = '';
23       data.forEach((emp, i) => {
24         const row = `
25           <tr>
26             <td>${i + 1}</td>
27             <td>${emp.name}</td>
28             <td>${emp.email}</td>
29             <td>${emp.department}</td>
30             <td>${emp.salary}</td>
31             <td>
32               <button class="btn btn-info btn-sm" onclick="viewEmployee('${emp.id}')">View</button>
33               <button class="btn btn-warning btn-sm" onclick="editEmployee('${emp.id}')">Edit</button>
34               <button class="btn btn-danger btn-sm" onclick="deleteEmployee('${emp.id}')">Delete</button>
35             </td>
36           </tr>`;
37       tbody.insertAdjacentHTML('beforeend', row);
38     });
39   });
40   .catch(err => showFormAlert('Error loading employees.'))
41   .finally(() => toggleLoader(false));
42 }
43
```

5. Form Alert Display

```
43
44 // Show alert message
45 function showFormAlert(message) {
46   const alertBox = document.getElementById('formAlert');
47   alertBox.textContent = message;
48   alertBox.classList.remove('d-none');
49 }
50
51 // Clear alert
52 function clearFormAlert() {
53   const alertBox = document.getElementById('formAlert');
54   alertBox.textContent = '';
55   alertBox.classList.add('d-none');
56 }
```

- showFormAlert(msg): Shows error messages in an alert box.
- clearFormAlert(): Hides the alert box and clears text.

6. Handle Add/Update Form Submit

- Prevents the page from reloading (e.preventDefault()).
- Builds the employee object from form inputs.
- Validates form (frontend).
- Sends POST (new) or PUT (update) request to backend.
- Shows validation errors from backend if any.

Employee Management System

- On success, reloads the employee table and resets the form.

```
58 // Handle form submit for adding/updating employee
59 function handleFormSubmit(e) {
60   e.preventDefault();
61   clearFormAlert();
62
63   const employee = {
64     name: document.getElementById('name').value.trim(),
65     email: document.getElementById('email').value.trim(),
66     department: document.getElementById('department').value.trim(),
67     salary: parseFloat(document.getElementById('salary').value)
68   };
69
70   if (!validateForm(employee)) return;
71
72   const method = editingId ? 'PUT' : 'POST';
73   const url = editingId ? `${apiUrl}/${editingId}` : apiUrl;
74
75   toggleLoader(true);
76   fetch(url, {
77     method: method,
78     headers: { 'Content-Type': 'application/json' },
79     body: JSON.stringify(employee)
80   })
81     .then(async res => {
82       if (!res.ok) {
83         const error = await res.json();
84         if (typeof error === 'object') {
85
86           // Show field errors
87           Object.keys(error).forEach(field => {
88             const errEl = document.getElementById(field + 'Error');
89             if (errEl) errEl.textContent = error[field];
90           });
91         } else {
92           showFormAlert(error);
93         }
94       } else {
95         showFormAlert(error);
96       }
97       throw new Error("Validation failed");
98     })
99     .then(res => {
100       fetchEmployees();
101       resetForm();
102     })
103     .catch(err => {
104       if (err.message !== "Validation failed") showFormAlert("Something went wrong.");
105     })
106     .finally(() => toggleLoader(false));
107 }
```

7.Frontend Form Validation

- Checks if all fields are filled properly.
- Checks if email format is valid.
- Checks if salary is a positive number.
- Displays error messages near respective form fields.

Employee Management System

```
108 // Validate the employee form (frontend validation)
109 function validateForm(emp) {
110   let valid = true;
111   document.getElementById('nameError').textContent = emp.name ? '' : 'Name is required';
112   document.getElementById('emailError').textContent =
113     !emp.email ? 'Email is required' :
114     !/\S+@\S+\.\S+/.test(emp.email) ? 'Invalid email format' : '';
115   document.getElementById('departmentError').textContent = emp.department ? '' : 'Department is required';
116   document.getElementById('salaryError').textContent = emp.salary > 0 ? '' : 'Salary must be positive';
117
118   if (!emp.name || !emp.email || !emp.department || emp.salary <= 0 || !/\S+@\S+\.\S+/.test(emp.email)) {
119     valid = false;
120   }
121   return valid;
122 }
```

8. Edit Employee

```
4 //Edit employee
5 function editEmployee(id) {
6   fetch(`${apiUrl}/${id}`)
7     .then(res => res.json())
8     .then(emp => {
9       document.getElementById('name').value = emp.name;
10      document.getElementById('email').value = emp.email;
11      document.getElementById('department').value = emp.department;
12      document.getElementById('salary').value = emp.salary;
13
14      editingId = id;
15      document.getElementById('submitBtn').textContent = 'Update Employee';
16      document.getElementById('formTitle').textContent = 'Edit Employee';
17    });
18 }
```

- Fills the form with the employee's data for editing.
- Changes the form title and button to “Update Employee”.

8.Reset Form

```
140 //Reset Form
141 function resetForm() {
142   document.getElementById('employeeForm').reset();
143   document.getElementById('submitBtn').textContent = 'Add Employee';
144   document.getElementById('formTitle').textContent = 'Add Employee';
145   editingId = null;
146
147   ['nameError', 'emailError', 'departmentError', 'salaryError'].forEach(id => {
148     document.getElementById(id).textContent = '';
149   });
150   clearFormAlert();
151 }
```

- Clears the form after adding or updating an employee.
- Resets button and title to "Add Employee".
- Clears validation messages and alert box.

10. Delete Employee

- Asks confirmation before deleting

Employee Management System

- Sends DELETE request to backend.
- Refreshes the employee list.

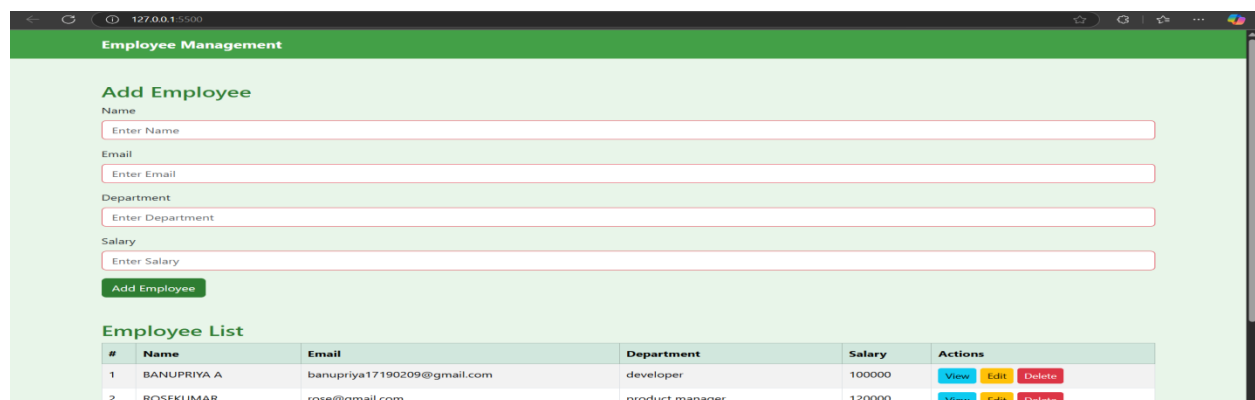
```
152
153 //Delete employee
154 function deleteEmployee(id) {
155   if (confirm('Are you sure you want to delete this employee?')) {
156     toggleLoader(true);
157     fetch(`${apiUrl}/${id}`, { method: 'DELETE' })
158       .then(() => fetchEmployees())
159       .catch(() => showFormAlert('Error deleting employee.'))
160       .finally(() => toggleLoader(false));
161   }
162 }
163
```

11. View Employee Details (Modal)

```
163
164 //View employee details
165 function viewEmployee(id) {
166   fetch(`${apiUrl}/${id}`)
167     .then(res => res.json())
168     .then(emp => {
169       document.getElementById('viewName').textContent = emp.name;
170       document.getElementById('viewEmail').textContent = emp.email;
171       document.getElementById('viewDepartment').textContent = emp.department;
172       document.getElementById('viewSalary').textContent = emp.salary;
173
174       const modal = new bootstrap.Modal(document.getElementById('viewModal'));
175       modal.show();
176     });
177 }
178
```

- Gets full employee details from backend by ID.
- Fills and opens a Bootstrap modal to display them.

MY FRONTEND VIEW



Employee Management

Add Employee

Name

Email

Department

Salary

Employee List

#	Name	Email	Department	Salary	Actions
1	BANUPRIYA A	banupriya17190209@gmail.com	developer	100000	View Edit Delete
2	ROSEKUMAR	rose@gmail.com	product manager	120000	View Edit Delete

Employee Management System

VIEW EMPLOYEE

The screenshot shows a web application interface for viewing employee details. On the left, there is a form with input fields for 'Email', 'Department', and 'Salary', each with a placeholder 'Enter [field name]'. Below these fields is a green 'Add Employee' button. In the center, a modal window titled 'Employee Details' is open, displaying the following information: Name: BANUPRIYA A, Email: banupriya17190209@gmail.com, Department: developer, and Salary: 100000. A green 'Close' button is at the bottom right of the modal. Below the modal, there is a section titled 'Employee List' containing a table with 5 rows of employee data.

#	Name	Email	Department	Salary	Actions
1	BANUPRIYA A	banupriya17190209@gmail.com	developer	100000	View Edit Delete
2	ROSEKUMAR	rose@gmail.com	product manager	120000	View Edit Delete
3	SHASHINI.R	shashini@gmail.com	product manager	120000	View Edit Delete
4	YASH KUMAR.R	yash@gmail.com	scrum master	100000	View Edit Delete
5	KAMALI	kamali@gmail.com	mean stack developer	100000	View Edit Delete

EDIT EMPLOYEE

The screenshot shows a web application interface for editing employee details. The form is titled 'Edit Employee' and contains input fields for 'Name', 'Email', 'Department', and 'Salary'. The values entered are: Name: BANUPRIYA A, Email: banupriya17190209@gmail.com, Department: developer, and Salary: 100000. A green 'Update Employee' button is at the bottom of the form. Below the form, there is a section titled 'Employee List' containing a table with 5 rows of employee data.

#	Name	Email	Department	Salary	Actions
1	BANUPRIYA A	banupriya17190209@gmail.com	developer	100000	View Edit Delete
2	ROSEKUMAR	rose@gmail.com	product manager	120000	View Edit Delete
3	SHASHINI.R	shashini@gmail.com	product manager	120000	View Edit Delete
4	YASH KUMAR.R	yash@gmail.com	scrum master	100000	View Edit Delete
5	KAMALI	kamali@gmail.com	mean stack developer	100000	View Edit Delete

Employee Management System

DELETE EMPLOYEE

The screenshot shows a web application interface for managing employees. A modal dialog box is displayed in the center, asking for confirmation to delete an employee. The dialog has a title "127.0.0.1:5500 says" and the text "Are you sure you want to delete this employee?". It includes "OK" and "Cancel" buttons. In the background, the application form is visible with fields for "Email", "Department", and "Salary", each with a placeholder "Enter [field name]". Below the form is a green "Add Employee" button. At the bottom, there is a table titled "Employee List" with 5 rows of employee data and an "Actions" column containing "View", "Edit", and "Delete" buttons for each row.

#	Name	Email	Department	Salary	Actions
1	BANUPRIYA A	banupriya17190209@gmail.com	developer	100000	View Edit Delete
2	ROSEKUMAR	rose@gmail.com	product manager	120000	View Edit Delete
3	SHASHINI.R	shashini@gmail.com	product manager	120000	View Edit Delete
4	YASH KUMAR.R	yash@gmail.com	scrum master	100000	View Edit Delete
5	KAMALI	kamali@gmail.com	mean stack developer	100000	View Edit Delete

CONCLUSION:

All the requirements are done in the above application **“EMPLOYEMENT MANAGEMENT SYSTEM”** “USING SPRING WEBFLUX.